Enterprise AI Ontologies

A Practitioner's Guide to Building a Living Meaning Layer

RDF | SKOS | OWL | SHACL | Catalog Synchronization | Refresh Flywheels

NIDHI VICHARE

June 2026

Companion guide for The Meaning Layer: Who Governs the AI That Defines Your Data

> **Executive position**
>
> Do not ask an LLM to invent enterprise meaning at runtime. Use AI to accelerate discovery, mapping, and change proposals. Keep approved business meaning in a governed semantic layer with stable identifiers, explicit definitions, validation rules, ownership, versioning, and catalog propagation.

This guide is intentionally pragmatic. It starts with a lightweight business vocabulary and only adds formal OWL semantics where inference produces measurable value. It separates two purchasing decisions that are frequently conflated: the catalog layer used by teams for discovery and stewardship, and the ontology runtime used for machine-readable semantics, validation, and reasoning.

Guide map

Section	Topic	Outcome
1	What you are building	A living meaning layer, not a static glossary
2	Standards and modeling choices	RDF, SKOS, OWL, SHACL, PROV-O
3	Build the minimum viable ontology	Concept selection, URIs, examples and controls
4	Run the executable starter kit	Code, validation, refresh proposals and context API
5	Design the refresh flywheel	How the ontology stays current
6	Propagate meaning across the enterprise	Catalogs, lineage, semantic models and AI applications
7	Options analysis	Atlan, OpenMetadata, Alation and ontology-native tools
8	Scale through an operating model	Release process, accountability, KPIs and 12-week plan

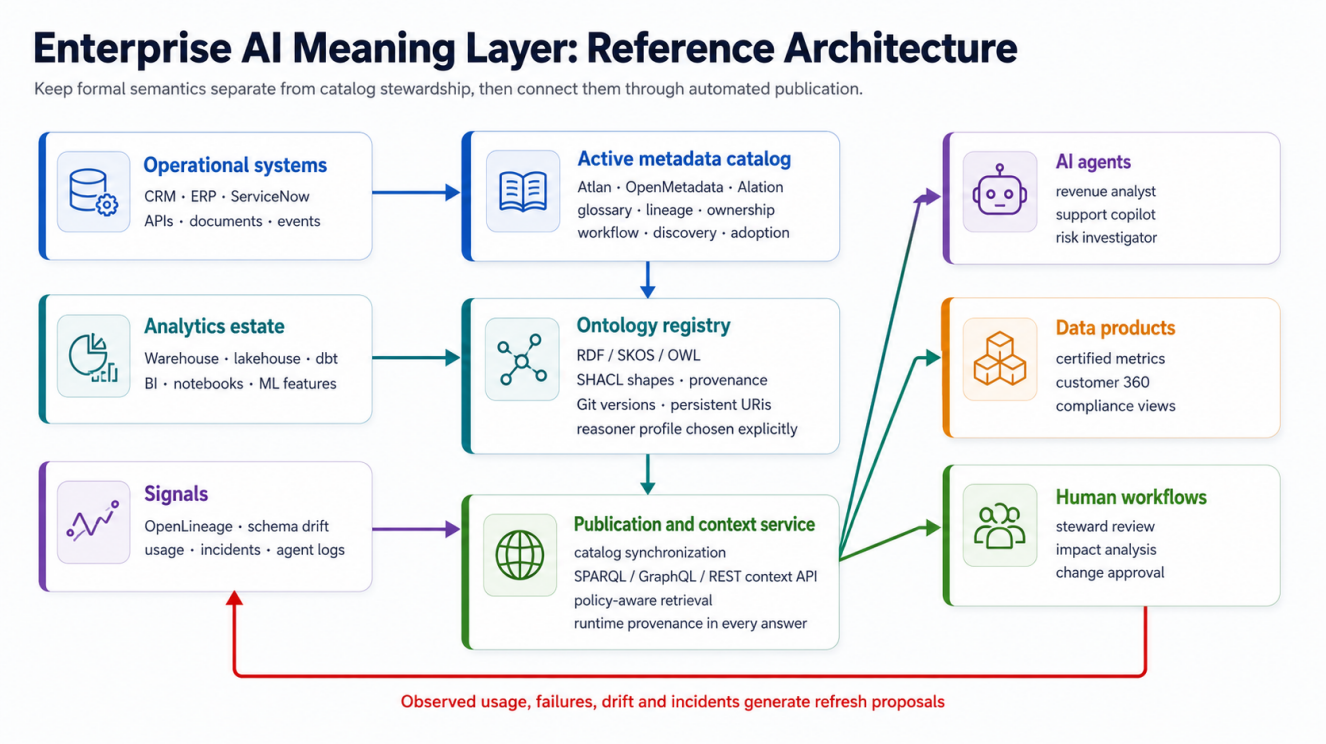
1. What you are building

Enterprise AI needs a controlled source of meaning. Data catalogs, metric stores, semantic layers, knowledge graphs, policies, and LLM prompts each solve part of the problem. An ontology connects those components by assigning stable identifiers to business concepts and by making their relationships, definitions, owners, controls, and implementations explicit.

1.1 Separate the layers

Layer	Purpose	Primary business outcome
Glossary	Human-readable definition of terms such as Customer, Active Customer, Net Revenue and Product.	Alignment and search.
Taxonomy	Hierarchies and classification schemes.	Navigation, tagging and domain organization.
Ontology	Formal concepts, relationships, constraints and selected inference rules.	Machine-readable meaning and controlled reasoning.
Semantic layer	Executable metrics and entity logic exposed to BI and AI applications.	Consistent calculation and query behavior.
Catalog	Stewardship interface, discovery, lineage, workflows and metadata propagation.	Adoption across teams.
Graph runtime	Store, query, validate and optionally reason over RDF.	Runtime context and machine integration.

1.2 Use a layered architecture



Reference architecture for an enterprise AI meaning layer.

The recommended pattern is a thin governed ontology registry at the center. Operational signals and analytics signals create proposals. Stewards approve releases. A publication layer synchronizes approved terms to catalogs, data products, semantic models, and human workflows. The catalog remains the primary collaboration surface; the ontology registry remains the portable semantic source of truth.

Layer	Primary purpose	Recommended approach
Business vocabulary	Definitions, synonyms, taxonomies, ownership	Model with SKOS and synchronize into the enterprise catalog
Formal ontology registry	Stable identifiers, relationships, policy links, releases	Store RDF/Turtle in Git with controlled URIs and versioned releases
Validation layer	Prevent incomplete or unsafe meanings from being published	Enforce SHACL controls in CI/CD
Graph runtime	Query approved meaning and optionally perform reasoning	Stardog, GraphDB, Jena/Fuseki, RDF4J, or Ontop
Catalog collaboration layer	Discovery, stewardship, workflow, lineage, propagation	Evaluate Atlan, OpenMetadata, and Alation
AI context service	Supply approved meaning to agents and copilots	Expose a governed API or MCP-style tool
Refresh flywheel	Keep the ontology current	Convert drift, usage, quality incidents, and AI failures into reviewed change proposals

2. Standards and modeling choices

2.1 SKOS-first, SHACL-first, selective-OWL strategy

Begin with 25 to 75 high-value concepts tied to a funded business use case. Give every concept a stable URI, preferred label, definition, steward, lifecycle state, review cadence, and mapped implementation. Require deterministic calculation logic and source assets for governed metrics. Add OWL relationships and bounded reasoning only where they improve a decision, control, impact analysis, search result, or AI answer. Deprecate concepts rather than deleting them, so older dashboards and agent interactions remain traceable.

2.2 RDF: the triple

Every fact in the ontology is a triple: subject, predicate, object. In Turtle syntax:

```
ex:CustomerTerm a skos:Concept, ex:EntityConcept ;
  skos:inScheme ex:EnterpriseBusinessVocabulary ;
  skos:prefLabel "Customer"@en ;
  skos:altLabel "Client"@en, "Buyer"@en ;
  skos:definition "A party with an active or historical commercial
    relationship with the enterprise."@en ;
  ex:steward "Customer Data Council" ;
  ex:status "Approved" ;
  ex:lastReviewedAt "2026-05-20"^^xsd:date ;
  ex:reviewCadenceDays 90 ;
  ex:implementedByAsset ex:CustomerMasterTable ;
  ex:governedByPolicy ex:CustomerPIIPolicyTerm .
```

This is not a glossary entry. It is a governed meaning artifact: definition, steward, lifecycle status, review cadence, implementing asset, and governing policy, all queryable and validatable.

2.3 SHACL: the governance gate

SHACL shapes enforce that every concept meets governance requirements before publication:

```
ex:GovernedConceptShape a sh:NodeShape ;
  sh:targetClass skos:Concept ;
  sh:property [
    sh:path skos:prefLabel ;
    sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype rdf:langString ;
    sh:message "Every governed concept must have exactly
      one preferred label with a language tag." ;
  ] ;
  sh:property [
    sh:path ex:steward ;
    sh:minCount 1 ;
    sh:message "Every governed concept requires
      an accountable steward." ;
  ] ;
  sh:property [
    sh:path ex:status ;
    sh:minCount 1 ;
    sh:in ( "Draft" "Approved" "Deprecated" ) ;
    sh:message "Lifecycle status must be Draft,
      Approved, or Deprecated." ;
  ] .
```

When invalid data is submitted, SHACL produces a structured violation report:

```
SHACL conforms: False
Triples validated: 177
Results (6):
  Lifecycle status must be Draft, Approved, or Deprecated.
  A governed metric must link to at least one implementing data asset.
  A governed metric requires deterministic calculation logic.
  Each data asset requires an accountable owner or steward.
  Each data asset requires an asset type.
  Every governed concept requires a positive review cadence.
```

3. Build the minimum viable ontology

The starter kit ontology contains four governed concepts:

Concept	Meaning	Steward	Linked asset
Customer	A party with an active or historical commercial relationship with the enterprise	Customer Data Council	Customer master table
Active Customer	A customer with at least one qualifying transaction in the trailing 12 months	Growth Analytics	Customer 360 view
Net Revenue	Recognized gross revenue less discounts, returns, credits, and rebates	Finance Analytics	Revenue mart
Customer PII Policy	Customer-identifying fields must be masked outside approved workflows	Privacy Office	Policy concept

For each concept, require: stable URI, preferred name, synonyms, business definition, steward, lifecycle status, review cadence, calculation logic (if metric), mapped catalog asset, dependent concepts, and policy links.

4. Run the executable starter kit

```
git clone https://github.com/nvichare/meaning-layer-starter-kit
cd meaning-layer-starter-kit
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Command	What it does	Expected result
`make validate`	Validate governed data against SHACL shapes	167 triples validated, all pass
`make validate-invalid`	Validate deliberately broken data	6 governance violations caught
`make query`	Look up Net Revenue definition	Returns definition, formula, steward, catalog ID
`make refresh`	Find stale concepts	Active Customer flagged as overdue for review
`make export`	Export glossary to CSV	4 catalog-ready terms exported
`make api`	Start agent context API	FastAPI at localhost:8000
`make test`	Run full test suite	75 tests pass in ~3 seconds

4.1 The context API

Agents call the governed API before they act:

```
GET /concepts/Net%20Revenue

{
  "label": "Net Revenue",
  "definition": "Recognized gross revenue less contractual
    discounts, returns, credits, and rebates.",
  "status": "Approved",
  "steward": "Finance Analytics",
  "calculation_logic": "SUM(gross_revenue - discounts
    - returns - credits - rebates)",
  "implemented_by_assets": [{
    "catalog_id": "snowflake.finance.revenue_mart",
    "source_system": "Snowflake.FINANCE",
    "asset_type": "Table"
  }]
}
```

A query for a non-existent concept returns 404. The agent should escalate to a steward, not manufacture a definition.

5. Design the refresh flywheel

> **The design rule**
>
> Use deterministic systems to detect objective changes. Use AI to interpret the meaning, rank the business impact, draft ontology changes, and route the proposal to the correct steward. Do not allow an LLM to silently rewrite the enterprise ontology.

The refresh flywheel has three phases:

Phase	Signal types
Phase 1: Foundation	Review overdue, schema drift, new dbt models, new semantic metrics, lineage changes, quality incidents
Phase 2: Usage learning	Catalog no-result searches, agent low-confidence responses, repeated user clarifications, heavily used but weakly governed concepts
Phase 3: Rationalization	Dashboard definition disagreements, unused concept deprecation

Every signal produces a governed proposal, not a silent change:

```
Detect objectively (deterministic)
    |
Generate AI proposal (interpret, classify, draft)
    |
Attach evidence (source event, impacted assets, confidence)
    |
Run SHACL validation (governance gate)
    |
Obtain steward approval (human ratification)
    |
Merge through Git (version control)
    |
Publish versioned release (catalog sync, context API refresh)
```

6. Propagate meaning across the enterprise

The ontology exports to any catalog that accepts CSV import. The starter kit includes adapters for:

Catalog	Adapter	Mode
Atlan	`make sync-atlan`	Generates Business Graph import CSV
OpenMetadata	`make sync-openmetadata`	Dry-run JSON payloads (configure tenant for live)
Alation	Use `glossary_terms.csv`	Map columns to Alation glossary import

6.1 Vendor evaluation criteria

Capability	Required behavior
Stable concept URI	Must remain attached to the catalog term
Definition	Must preserve the approved business definition
Synonyms	Must improve discoverability
Steward	Must identify accountable ownership
Lifecycle	Must support draft, approved, and deprecated states
Review date	Must support recertification workflows
Asset links	Must connect definitions to physical or semantic assets
API or automation	Must allow synchronization without manual re-entry

7. Options analysis

Atlan

Strong managed collaboration and propagation layer with Business Graph, Metadata Propagator, and Playbooks. Pair with a portable RDF/SKOS/SHACL registry and a graph runtime when formal semantic validation or reasoning is required.

OpenMetadata

Strong open-source candidate with glossary hierarchies, synonyms, lineage APIs, and approval workflows. Its documentation describes canonical schemas alongside RDF/OWL models, SHACL shapes, and PROV-O. Validate the deployed version during proof of value.

Alation

Strong enterprise catalog with business-glossary, governance, lineage, and workflow capabilities. Use a separate ontology registry and runtime for formal RDF/OWL semantics unless tenant evaluation demonstrates required coverage.

Ontology-native tools

For modeling: Protege or WebProtege (open-source) or TopBraid EDG (enterprise). For runtime: Stardog, GraphDB, Apache Jena/Fuseki, Eclipse RDF4J, or Ontop.

8. Scale through an operating model

12-week rollout

Timing	Outcome
Weeks 1-2	Select one business use case, 25-75 concepts, stewards, competency questions, and measurable outcomes
Weeks 3-4	Create the SKOS vocabulary, URI rules, minimum relationships, and release discipline
Weeks 5-6	Map concepts to assets and metrics, add SHACL controls, and establish the CI gate
Weeks 7-8	Synchronize approved concepts into the selected catalog and enable workflows
Weeks 9-10	Connect one AI application and one analytics workflow to the governed context API
Weeks 11-12	Add refresh signals, measure KPIs, document operating decisions, and select the next domain

Success metrics

Metric	What to measure
AI grounding quality	Reduction in unsupported or inconsistent AI answers
Metric consistency	Reduction in conflicting dashboard calculations
Steward accountability	Percentage of critical concepts with owners and review dates
Change responsiveness	Time between schema drift and approved ontology update
Governance automation	Percentage of weak proposals rejected automatically
Adoption	Number of active users retrieving or contributing governed definitions

---

The enterprise that owns its meaning owns its future.

Companion guide for *The Meaning Layer: Who Governs the AI That Defines Your Data* by Nidhi Vichare.
Starter kit: github.com/nvichare/meaning-layer-starter-kit | MIT License